桥牌汇 BridgeHub · 部署手册

服务器部署 · 运维 · 故障排查

桥牌汇 BridgeHub · 2026 年 10 月

适用版本：**v1.8.2**（前后端一体 + Nginx 反代架构）

目标环境：Ubuntu 22.04 / 24.04 LTS（本文以 24.04 为准）

>

v1.8.2 更新要点（手机端显示修复，本版无安全/数据变更）

- 修复手机上**明手牌的花色被相邻牌遮住**：横排牌堆的重叠幅度改为按视口自适应（CSS `min()` 计算），

纵排（东/西座位）的角标改为**点数与花色同一行**；可点牌的高亮描边改为内嵌（`ring-inset`）不再外扩

- 手机端大牌点数字号 18px → 16px（仍大于 v1.7.x 的 12px；桌面 21px 不变），

13 张牌在 360–390 宽机型上不再溢出牌桌

>

v1.8.1 更新要点（打牌节奏）

- 收墩停顿由 3000ms 改为 **1000ms**（v1.8.0 曾为 3 秒，反馈偏慢），可用环境变量 `TRICK_CLEAR_MS` 覆盖（见 5.3 节）

- 服务端默认值与前端常量同步改为 `1000`，升级只需替换 `dist/` 与 `dist-server/`，**数据格式不变、无需迁移**

>

v1.8.0 更新要点

- 明手改为**首攻打出后**才摊牌（服务端 `dummyHand` 视图裁剪同步收紧，客户端无法提前拿到）

- 收墩停顿由 1100ms 改为 3000ms（v1.8.1 起再改为 1000ms），可用环境变量 `TRICK_CLEAR_MS` 覆盖（见 5.3 节）

- 牌面尺寸与点数/花色字号整体 **放大 50%**，同步重排手牌重叠间距

- 升级只需替换 `dist/` 与 `dist-server/`，**数据格式不变、无需迁移**

>

v1.7.0 更新要点

- 新增 `server/security.ts`：请求体限制、安全响应头、IP 限流、WebSocket 接入限流

- **务必**在新机器上确认 `TRUST_PROXY=1` 已生效（见 5.5 节），否则限流会误伤全体用户

- `/ws` 的 Nginx 转发头已补全（Nginx 的同名指令继承陷阱，见 5.5 节）

- 新增 IMP 计分支持，无需额外配置

>

v1.7.1 更新要点（建议所有 v1.7.0 实例升级）

- 修复**限流可被绕过**的缺陷：`clientIp()` 原先取 `X-Forwarded-For` 最左侧，

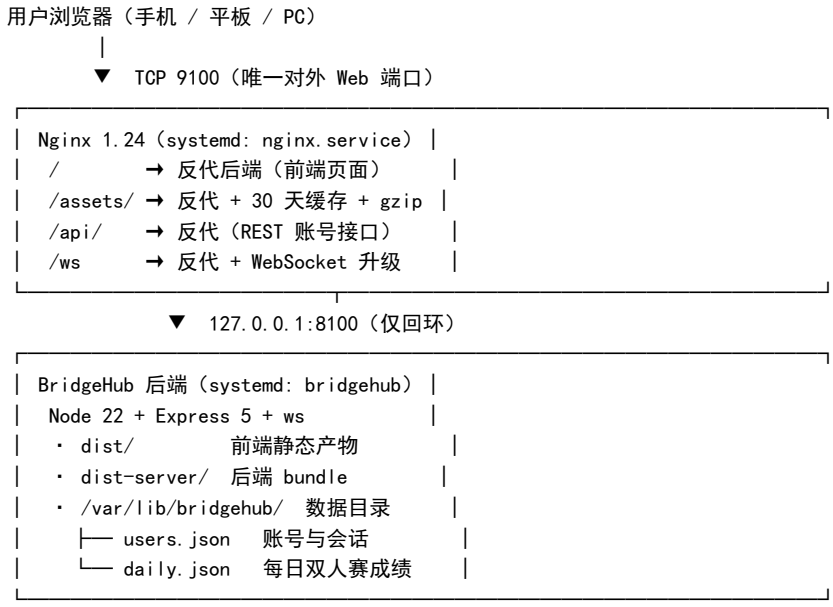
而该字段是「追加」语义，客户端伪造一个 XFF 就能换来全新配额，限流形同虚设

- 改为优先取 Nginx 无条件覆写的 `X-Real-IP`，XFF 仅作回落且取**最右侧**

- 新增 `TRUST_PROXY_HOPS`（默认 1）支持多级代理
- 仅服务端改动，**前端产物与数据格式不变**，升级无需迁移、不影响用户数据

---

1. 架构总览



对外防火墙（ufw）只放行两个端口：

端口	用途
22/tcp	SSH 管理
9100/tcp	Web 访问（Nginx）

后端的 8100 端口仅监听本机回环，外部不可直连。

---

2. 服务器要求

项目	最低	推荐
CPU	1 核	2 核
内存	1 GB	2 GB
磁盘	2 GB	10 GB
系统	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
网络	可访问 npmjs.org 与 nodejs.org（仅安装时需要）	同左

---

3. 全新部署（在一台干净的 Ubuntu 上）

3.1 准备发布包（在开发机上执行）

```
cd app
npm install                # 首次需要
npm run build              # 前端：tsc 类型检查 + vite 构建到 dist/
npm run build:server       # 后端：esbuild 打包到 dist-server/index.js

# 组装发布包
rm -rf release bridgehub-release.tar.gz && mkdir -p release
cp -r dist release/dist
cp -r dist-server release/dist-server
cp deploy/bridgehub.service release/
cat > release/package.json <<'EOF'
{
  "name": "bridgehub-server",
  "private": true,
  "type": "module",
  "scripts": { "start": "node dist-server/index.js" },
  "dependencies": { "express": "^5.2.1", "ws": "^8.21.3" }
}
EOF
tar -czf bridgehub-release.tar.gz -C release .
```

3.2 上传到服务器

```
scp bridgehub-release.tar.gz deploy/setup-bridgehub.sh deploy/setup-nginx.sh deploy/nginx.conf user@<服务器IP>:/tmp/
# 注意：nginx.conf 上传为目标脚本期望的文件名
ssh user@<服务器IP> 'mv /tmp/nginx.conf /tmp/bridgehub-nginx.conf'
```

3.3 执行部署脚本（服务器上，需 sudo）

```
# 第一步：应用本体（装 Node 22 → 解压到 /opt/bridgehub/app → 装依赖 → 注册 systemd）
sudo bash /tmp/setup-bridgehub.sh

# 第二步：Nginx 反向代理（装 nginx → 站点配置 listen 9100 → 防火墙放行 9100）
sudo bash /tmp/setup-nginx.sh
```

两个脚本都是**幂等**的：重复执行只会更新到最新发布包并重启服务。

3.4 验证

```
curl http://127.0.0.1:9100/api/health
# 期望输出：{"ok":true,"name":"BridgeHub","time":"..."}
```

浏览器访问 `http://<服务器IP>:9100/` 即可。

---

4. 日常更新版本

代码更新后，只需在开发机重新执行 3.1（打包）→ 3.2（上传）→ 然后服务器上：

```
sudo bash /tmp/setup-bridgehub.sh
```

脚本会重新解压、安装依赖并 `systemctl restart bridgehub`。Nginx 配置无需变动。
用户数据不受影响（数据在 `/var/lib/bridgehub/`，不在应用目录内）。

---

5. 运维操作

5.1 服务管理

```
sudo systemctl status bridgehub      # 后端状态
sudo systemctl restart bridgehub     # 重启后端
sudo journalctl -u bridgehub -f      # 实时日志
sudo nginx -t                        # 校验 nginx 配置
sudo systemctl reload nginx          # 热加载 nginx
```

5.2 数据备份与迁移

只需备份数据目录中的两个文件：

```
sudo cp /var/lib/bridgehub/users.json /var/lib/bridgehub/daily.json ~/backup/
```

迁移到新服务器：先完成全新部署，再把这两个文件覆盖到 `/var/lib/bridgehub/`， `sudo systemctl restart bridgehub` 即可。

5.3 环境变量（改 `/etc/systemd/system/bridgehub.service` 后需 `daemon-reload` + `restart`）

变量	默认值	说明
`PORT`	`8100`	后端监听端口（配合 nginx 无需改）
`BRIDGEHUB_DATA`	`server/data`	数据目录，生产为 `/var/lib/bridgehub`
`BOT_BID_MS` / `BOT_PLAY_MS` / `TRICK_CLEAR_MS`	`650` / `750` / `1000`	AI 行动节奏与收墩停顿（毫秒）。`TRICK_CLEAR_MS` 为**一墩四张收齐后停留多久再清台**，v1.8.1 起默认 1000（v1.8.0 为 3000，更早为 1100），兼顾"看清本墩"与"不拖节奏"；压测或想改变节奏时可按需调整
`TRUST_PROXY`	未设置（不信任）	**反代部署必须设为 `1`**，否则客户端 IP 全为 `127.0.0.1`，限流失效且误伤全体用户。前提：应用只能通过会覆写 `X-Real-IP` 的反向代理对外暴露（本项目 Nginx 配置已满足），否则不要开启
`TRUST_PROXY_HOPS`	`1`	可信代理层数（v1.7.1 起）。决定了 `X-Forwarded-For` 从**右往左**数第几段才是真实客户端 IP；多级代理链按需调大
`BEHIND_HTTPS`	未设置（不发送）	设为 `1` 后下发 `Strict-Transport-Security` 头。确认全站 HTTPS 后再开启

`JSON_LIMIT`	`32kb`	REST 请求体大小上限，超限返回 413
`AUTH_RATE_MAX` / `AUTH_RATE_WINDOW_MS`	`20` / `300000`（5 分钟）	注册/登录等认证接口的单 IP 配额
`API_RATE_MAX` / `API_RATE_WINDOW_MS`	`120` / `60000`（1 分钟）	其余 REST 接口的单 IP 配额

5.4 内置安全加固（v1.7.0 起，v1.7.1 修正限流键取值）

后端新增 `server/security.ts`，**无需额外依赖**，实现以下防护：

防护项	实现
请求体限制	`express.json({ limit })`，默认 32KB，超限返回 HTTP 413
安全响应头	`X-Frame-Options`、`X-Content-Type-Options`、`Content-Security-Policy`、`Referrer-Policy`、`Permissions-Policy`、`COOP/CORP`
技术栈隐藏	移除 `X-Powered-By`
HSTS	仅在 `BEHIND_HTTPS=1` 时下发，避免误配导致站点被强制跳转 HTTPS 而打不开
REST 限流	按 IP 的滑动窗口；超限进入指数退避惩罚期（30 秒起，最长 30 分钟），响应带 `Retry-After` 与 `X-RateLimit-*`
WebSocket 限流	单 IP 每分钟最多 30 条新连接、10 次认证失败；超出直接拒绝握手
账号枚举防护	登录失败一律返回「用户名或密码不正确」，不区分账号是否存在
限流键防伪造（v1.7.1）	优先取 Nginx 覆写的 `X-Real-IP`；`X-Forwarded-For` 仅作回落且取**最右侧**（代理追加的真实 IP），而非可被客户端伪造的最左侧

> **重要（一）**：上述所有限流均以**真实客户端 IP** 为依据。后端部署在 Nginx 之后时，必须同时满足两个条件才能取到真实 IP，缺一不可：
>
> 1. Nginx 转发了 `X-Real-IP`（无条件覆写，推荐）或 `X-Forwarded-For`——且 **`/ws` 块内必须重复声明**（Nginx 规定：location 内一旦自定义 `proxy_set_header`，就不再继承 server 级同名指令，这是最容易踩的坑）；
> 2. 后端设置了 `TRUST_PROXY=1`。

> 否则所有请求的 IP 都会被识别为 `127.0.0.1`，全体用户挤在同一个配额桶里——任何一个人连续输错密码都会导致所有人都登录不上。

> **重要（二）**：v1.7.0 的实现有缺陷，已在 **v1.7.1** 修复。`X-Forwarded-For` 是**追加**语义，形如 `<客户端伪造值>, <代理追加的真实IP>`；旧版本取**最左侧**，等于把限流键交给攻击者——请求时换个 `X-Forwarded-For` 就能立刻拿到满额配额，限流形同虚设。
>
> 因此 v1.7.0 实例**必须升级**，升级后用 5.5 节第 5 条命令复核。

5.5 验收安全配置是否生效

```
# 1) 安全响应头是否下发（应看到 X-Frame-Options 等，且不出现 X-Powered-By）
curl -sSI http://127.0.0.1:9100/api/health | grep -iE 'x-frame|content-security|x-ratelimit|x-powered-by'

# 2) 请求体限制（超限应返回 413）
curl -sS -X POST http://127.0.0.1:9100/api/register \
  -H 'Content-Type: application/json' \
  -d "{\"username\":\"$(head -c 40000 /dev/zero | tr '\0' 'a')\",\"password\":\"x\"}"

# 3) 认证限流（第 21 次应返回 429）
for i in $(seq 1 21); do
  curl -sS -o /dev/null -w "%{http_code} " -X POST http://127.0.0.1:9100/api/login \
    -H 'Content-Type: application/json' -d '{"username":"nosuchuser","password":"bad"}'
done; echo

# 4) 反代后能否取到真实客户端 IP
sudo systemctl show bridgehub -p Environment

# 5) 限流键是否防伪造（v1.7.1 起必须此项通过）
#    连续请求同一接口，中途插入伪造的 X-Forwarded-For，配额剩余量应**继续递减**；
#    若伪造后剩余量跳回满额，说明仍在取最左侧，限流可被绕过，需立即复核版本与配置。
for i in 1 2 3; do
  curl -sS -o /dev/null -D - http://127.0.0.1:9100/api/health | grep -i x-ratelimit-remaining
done
curl -sS -o /dev/null -D - -H 'X-Forwarded-For: 203.0.113.66' http://127.0.0.1:9100/api/health \
  | grep -i x-ratelimit-remaining
```

5.6 卸载

```
sudo systemctl disable --now bridgehub
sudo rm /etc/systemd/system/bridgehub.service
sudo rm -f /etc/nginx/sites-enabled/bridgehub /etc/nginx/sites-available/bridgehub
sudo systemctl reload nginx
sudo rm -rf /opt/bridgehub /var/lib/bridgehub   # 数据将一并删除，先备份！
```

---

6. 故障排查

现象	排查步骤

浏览器打不开 9100	① `sudo ufw status` 确认 9100 放行；② `sudo systemctl status nginx`；③ 服务器本地 `curl 127.0.0.1:9100/api/health`
页面 502 Bad Gateway	后端挂了：`sudo systemctl status bridgehub`、`sudo journalctl -u bridgehub -n 50`
能开页面但联机对战不更新	WebSocket 没通：检查 nginx 站点里 `/ws` 段的 `Upgrade/Connection` 头是否齐全；浏览器 DevTools → Network → WS 看握手是否 101
深链接（如 /event/daily）白屏	前端 `vite.config.ts` 的 `base` 必须是 `'/'`（相对路径 `./` 会导致深层路由资源 404），改后重新打包部署
注册/登录全部失败	检查数据目录权限：`sudo ls -l /var/lib/bridgehub` 应属 `www-data`；`sudo chown -R www-data:www-data /var/lib/bridgehub`
登录提示「操作过于频繁」（429）	① 确认是否真的在高频重试，等待惩罚期结束；② 若**所有人**都被限，说明后端只看到 `127.0.0.1`——检查 `TRUST_PROXY=1` 与 Nginx `/ws`/全局转发头（见 5.5 节）
联机对战连不上，且日志提示「连接过于频繁」	WebSocket 接入限流触发：检查是否有异常客户端刷连接；确认 Nginx `/ws` 块已重复声明 `X-Forwarded-For`
请求体过大被拒（413）	正常行为，默认上限 32KB。确需放宽时调 `JSON_LIMIT` 环境变量

换 X-Forwarded-For 就能绕过限流	说明仍在取 XFF 最左侧——请升级到 v1.7.1（见 7.3）；若已是 v1.7.1，检查 Nginx 是否覆写了 X-Real-IP		
每日赛成绩丢失	成绩在 daily.json，服务重启不丢；检查是否有备份被误覆盖		
SSH 突然登不上（认证卡住）	高频连接可能触发临时锁定，等待 2–5 分钟再试；确认账号未被 faillock 锁定（sudo faillock --user <名> --reset）		
端口被占用	`sudo ss -tlnp \	grep -E '9100\	8100'` 找出占用进程后处理

7. 安全说明

7.1 已实施的防护

- ☐ 密码使用 **scrypt**（随机盐 + timingSafeEqual 定时安全比较）存储，数据文件不落明文密码
- ☐ REST 请求体限制 32KB，超限返回 413
- ☐ 下发安全响应头（防点击劫持、MIME 嗅探、XSS、Spectre 类攻击），隐藏技术栈指纹
- ☐ 按 IP 的滑动窗口限流 + 指数退避惩罚，覆盖 REST 与 WebSocket
- ☐ 登录失败文案统一，防止账号枚举
- ☐ 运行时数据目录（app/data/）已加入 .gitignore，不会被提交进版本库

7.2 仍需人工处理的事项

项目	建议
传输加密	对外公网开放前务必上 HTTPS（Let's Encrypt），并设 BEHIND_HTTPS=1 下发 HSTS
历史泄露数据	若早期版本的 users.json 曾被提交过仓库，需用 git filter-repo / BFG 清除历史记录，并强制已注册用户改密码
凭证管理	GitHub Token、服务器私钥、生产 users.json 均不得出现在聊天记录、日志或仓库中；轮换后立即吊销旧凭证

邮箱验证	当前无邮箱验证，开放注册存在被批量注册的风险，建议加校验或将注册改为邀请制
存储升级	JSON 文件存储为演示级实现，量上来后建议迁移 SQLite / PostgreSQL
数据备份	定期备份 `/var/lib/bridgehub/`（可加入 crontab，例如每天 02:17 打包到 `~/backup/`）

7.3 升级注意事项

升级到 v1.8.2（手机端明手牌角标遮挡修复）

1. 仅前端展示变化（`game-ui.tsx`、`index.css` 的 `.spread-*` 重叠规则），服务端无变化；数据格式不变，**无需迁移**
2. 手机端大牌点数字号 18px → 16px；若线上有依赖旧字号的截图/验收脚本，需同步更新
3. 验收：用 360–390 宽的手机（或 DevTools 手机视口）进入牌桌，首攻后检查明手牌——每张牌的点数与花色应完整可见（横排看左上角、竖排看顶部一行），整排不溢出牌桌

升级到 v1.8.1（收墩停顿 3 秒 → 1 秒）

1. 仅一处行为参数变化：服务端默认 `TRICK_CLEAR_MS` 由 `3000` 改为 `1000`，前端 `GameTable.tsx` 常量同步改为 `1000`
2. 若线上 systemd 单元或环境文件里**显式写过** `TRICK_CLEAR_MS=3000`（或 `1100`），需同步改为 `1000` 或删除该行，否则环境变量优先级高于代码默认值，改动不会生效
3. 数据格式未变更，**无需迁移**；前后端建议一起更新（停顿由服务端计时器驱动，前端用于单机/本地模式）
4. 验收：进入任一牌桌打完一墩，确认四张牌收齐后约 1 秒（而非 3 秒）清台

升级到 v1.8.0（打牌体验改动）

1. 明手改为**首攻后才摊牌**：这是服务端视图裁剪的改动（`dummyHand` 仅在 `trick.length > 0` 时下发），客户端无法绕过；升级后**不要混用旧前端与新后端**（旧前端不会自行隐藏明手），前后端请一起更新
2. 收墩停顿 v1.8.1 起为 1000ms（本版曾为 3000ms）。若线上 systemd 单元里显式写过 `TRICK_CLEAR_MS=1100` 或 `TRICK_CLEAR_MS=3000`，需同步改掉或删除该行，否则环境变量优先级高于代码默认值，改动不会生效
3. 牌面放大 50%，前端产物 hash 会变化（`index-*.js` / `index-*.css`），部署后建议清理 `dist/assets/` 旧文件
4. 数据格式未变更，**无需迁移**，用户数据与战绩不受影响

5. 验收：进入任一牌桌，确认①叫牌刚结束、首攻未出时明手是牌背；②首攻打出后明手翻牌；

③一墩收齐后停留约 1 秒；④牌面明显变大且 13 张手牌不溢出屏幕

升级到 v1.7.1（含 v1.7.0 实例必做）

1. v1.7.0 的限流键取自 `X-Forwarded-For` 最左侧，**可被客户端伪造绕过**；v1.7.1 已修正取值规则
2. 升级方式：直接部署新的发布包并 `sudo systemctl restart bridgehub`；Nginx 配置与 `TRUST_PROXY=1` 无需改动
3. 升级后用 5.5 节第 5 条命令确认伪造 XFF 不再产生新的配额桶
4. 若有自建反向代理而非使用本项目 `deploy/nginx.conf`，请确保它会**覆写** `X-Real-IP`；

否则后端将回落使用 XFF 最右侧，需要按实际代理层数设置 `TRUST_PROXY_HOPS`

升级到 v1.7.0

1. 部署完成后**必须复核** `TRUST_PROXY=1` 是否随 systemd 单元生效（`systemctl show bridgehub -p Environment`）
2. 同步更新 Nginx 配置（`/ws` 块的转发头），然后 `nginx -t && systemctl reload nginx`
3. 用 5.5 节的四条命令验收；出问题按第 6 章排查
4. 数据文件格式未变更，**无需迁移**，用户数据不受影响

---

8. 当前生产实例信息（2026-10-14 部署）

当前共有两台生产实例，软件栈与目录结构完全一致（同一发布包 `bridgehub-release.tar.gz`），各自独立持有用户数据：

8.1 实例一：192.168.0.212

项目	值
服务器	`192.168.0.212`（Ubuntu 24.04.5 LTS, x86_64）
访问入口	`http://192.168.0.212:9100/`
Node.js	v22.23.3（`/usr/local`）
应用目录	`/opt/bridgehub/app`
数据目录	`/var/lib/bridgehub`
后端服务	`bridgehub.service`（www-data 运行，开机自启，崩溃自动重启）
反代服务	`nginx.service`，站点配置 `/etc/nginx/sites-available/bridgehub`

防火墙	ufw：仅放行 22/tcp 与 9100/tcp

8.2 实例二：192.168.20.8（局域网）

项目	值
服务器	192.168.20.8（主机名 ubuntu22-webchat，Ubuntu 24.04.5 LTS，x86_64，内核 6.8.0）
管理账号	administrator（uid 1000，**sudo 免密**；SSH 用户名为 administrator 而非 ubuntu）
访问入口	http://192.168.20.8:9100/（对外只开 9100，由 nginx 监听）
Node.js	v22.23.3（/usr/local/bin/node，已装好，部署脚本跳过下载）
应用目录	/opt/bridgehub/app
数据目录	/var/lib/bridgehub（users.json，v1.8.0 升级时 16 个账号，升级未受影响）
后端服务	bridgehub.service（www-data 运行，开机自启，Environment=PORT=8100 BRIDGEHUB_DATA=/var/lib/bridgehub TRUST_PROXY=1）
反代服务	nginx.service 1.24.0，站点配置 /etc/nginx/sites-available/bridgehub，listen 9100 → proxy_pass 127.0.0.1:8100
防火墙	ufw 未启用（inactive）。**注意**：node 监听 *:8100（全部网卡），局域网内可直连 8100 绕过 nginx；如需收紧，启用 ufw 后只放行 22 与 9100
版本轨迹	v1.6.0（2026-10-14 部署，静态资源堆积 16 个历史文件）→ v1.8.0（2026-10-23）→ v1.8.1（2026-10-26，收墩停顿 3 秒→1 秒）→ **v1.8.2（2026-10-26，手机端明手牌角标遮挡修复）**
升级前备份	/home/administrator/bridgehub-backups/（pre-v180-*.tar.gz、app-v160-*、nginx-bridgehub-pre-v180-*.conf、bridgehub.service-pre-v180-*、app-before-v1.8.1-*.tar.gz）

该实例无需安装 Node、无需联网拉依赖，setup-bridgehub.sh 全流程约 4 秒完成。

升级时同步替换了 nginx 配置（旧配置的 /ws 块缺少 X-Real-IP / X-Forwarded-For，会导致 WebSocket 侧限流把所有人识别成 127.0.0.1），并清理了 14 个历史静态资源（保留当前版本与上一版，避免已打开的页面白屏）。

8.3 实例三：82.156.202.73（公网）

项目	值
服务器	82.156.202.73（主机名 VM-0-11-ubuntu，Ubuntu 24.04.4 LTS，x86_64，内网 IP 10.2.0.11）
管理账号	ubuntu（sudo 需密码）
访问入口	http://82.156.202.73:9100/
Node.js	v22.23.3
应用目录	/opt/bridgehub/app
数据目录	/var/lib/bridgehub
后端服务	bridgehub.service（www-data 运行，开机自启）
反代服务	nginx.service 1.24.0，站点配置 /etc/nginx/sites-available/bridgehub
版本轨迹	v1.6.0（2026-10-19 部署）→ v1.7.0（2026-10-22）→ v1.7.1（2026-10-22，修正限流键取值）→ v1.8.0（2026-10-23，打牌体验）→ v1.8.1（2026-10-26，收墩停顿 3 秒→1 秒）→ **v1.8.2（2026-10-26，手机端明手牌角标遮挡修复）**
升级前备份	/home/ubuntu/bridgehub-backups/（含 pre-v170-*.tar.gz、pre-v180-*.tar.gz、app-v160-*、nginx-bridgehub-pre-*、app-before-v1.8.1-*.tar.gz）

该实例为**公网实例**，升级前请务必先看第 7.2 节「传输加密」——目前仍为 HTTP 明文访问。

8.4 上线验证记录（各实例均通过）

检查项	期望	结果
GET /api/health	返回 {"ok":true,"name":"BridgeHub",...}	通过
首页 GET /	HTTP 200，标题「桥牌汇 · BridgeHub」	通过
GET /ws（带 Upgrade 头）	返回 101 Switching Protocols	通过
安全响应头	8 项头齐备，X-Powered-By 不出现	通过
请求体超限	HTTP 413	通过
认证限流	第 21 次返回 429 并带 Retry-After	通过

账号枚举防护	存在/不存在账号返回完全一致的文案	通过
限流键防伪造（v1.7.1）	伪造 `X-Forwarded-For` 不产生新配额桶	通过
明手摊牌时机（v1.8.0）	首攻前明手为牌背，首攻落地后翻牌	通过
收墩停顿（v1.8.1）	一墩 4 张收齐后约 1 秒清台	通过
收墩停顿线上实测（v1.8.1）	82.156.202.73 与 192.168.20.8 各连测 3 墩，分别为 1001/1002/1001ms 与 1002/1002/1001ms	通过
手机端明手牌角标（v1.8.2）	手机视口（390/360 宽）8 桌抽样：上/下/左/右四个方位、大/小两种牌尺寸，点数与花色均完整可见，整排不溢出牌桌	通过
牌面尺寸（v1.8.0）	牌宽高与点数字号较 v1.7.x 增大约 50%，13 张手牌不溢出	通过

注意：两台实例的账号与每日赛成绩**互不相通**（数据各自存在本机 `/var/lib/bridgehub/`）。若需合并或迁移，按 5.2 节备份/恢复两个 JSON 文件即可。